# Introduction to Regular Expressions

**Presented by:**

Cristian Camilo Cortes Ortiz

# Table of contents

**Audience** 3
**Purpose** 3
**Introduction** 3
**Regular Expressions** 4
**Basic similarities** 4
. 4
\. 4
\n 4
\d 4
\D 4
\w 4
\W 5
\s 5
\S 5
**Flags** 5
\i 5
\g 5
**Limits** 5
\b 5
\B 5
^ 5
$ 5
**Quantifiers** 6
* 6
+ 6
? 6
{3} 6
{3-5} 6
**Characters sets** 6
[] 6
[^] 6
**Groups** 6
() 6
| 6

# Audience

This manual uses technical terminology and is intended for software development students or anyone interested in learning the fundamentals of pattern matching, data validation, and text processing with regular expressions (RegEx). It delves into concepts that provide a solid foundation for defining search patterns and enforcing complex business rules across programming languages and database environments.

# Purpose

The purpose of this manual is to provide readers with a clear understanding of the core concepts, syntax, and applications of regular expressions. It covers key topics such as character classes, quantifiers, anchors, group capturing, and character escaping, and shows how to build and apply patterns to validate formatted data (such as emails, passwords, and phone numbers) efficiently.

# Introduction

Regular Expressions, commonly referred to as RegEx or Regex, are sequences of characters that form search patterns used for string matching, validation, and parsing. Available across virtually all modern programming languages and relational database systems, regular expressions allow developers to enforce complex data rules and constraints without relying on lengthy conditional logic. From simple string searches to advanced input validation in back-end applications, regular expressions serve as a foundational tool in software engineering and automated text processing.

# Regular Expressions

In computer science, regular expressions (RegEx) are formal sequences of characters that define specific search patterns. By utilizing a precise syntax, they enable developers to perform advanced, segmented pattern matching and text validation across ASCII and ANSI character sets.

RegEx is supported universally across modern programming languages, text editors, and development tools to efficiently search, parse, and replace textual patterns. Historically, Unix and Linux command-line utilities such as the grep filter and the sed stream editor played a pivotal role in popularizing regular expressions and embedding them into standard engineering workflows.

# Basic similarities

### .

Allows you to search for any character that occupies a memory location, including a newline.

### \.

It allows you to “escape” the ‘period’ character using the \ and search for the “period” or another special character.

### \n

Allows you to search for the last character on a line.

### \d

Allows you to search for any character that is a digit (0–9).

### \D

Allows you to search for any character that is NOT a digit.

### \w

Allows you to search for any character in a word (a-z, A-Z, 0-9,_).

### \W

Allows you to search for any character that is NOT part of a word.

### \s

Allows you to search for any whitespace character (spaces, tabs, newlines).

### \S

Allows you to search for any character that is NOT a space.

## Flags

### \i

Allows you to search without distinguishing between uppercase and lowercase letters.

### \g

It allows you to search for all matches; without it, it returns only the first match.

## Limits

### \b

Allows you to search for characters within a word.

### \B

Allows you to search for characters that have NO word boundary.

### ^

Allows you to search for the beginning of a text string.

### $

Allows you to search for the end of a text string.

# Quantifiers

### *

Allows you to search for characters within a word.

### +

Allows you to search for characters that appear one or more times.

### ?

Allows you to search for characters that appear zero or one time.

### {3}

Allows you to search for characters that appear a specified number of times.

### {3-5}

Allows you to search for characters that appear a specified number of times.

# Characters sets

### []

Allows you to search for characters inside the brackets.

### [^]

Allows you to search for characters that are NOT inside the brackets.

# Groups

### ()

Allows you to search by grouping a specific sequence of characters.

### |

Allows you to search for one character or the other.